EXPERIMENT NO. 02

NAME OF EXPERIMENT: - To generate and represent standard discrete-time sequences using MATLAB.

OBJECTIVES:

1. To understand the concept of basic discrete-time signals used in Digital Signal Processing.

SOFTWARE: MATLAB V 7.7

PROGRAM:

```
clc;
clear;
close all;
n = -5:5;
% Unit Impulse
delta = (n==0);
% Unit Step
u = (n>=0);
% Ramp
r = n.*(n>=0);
figure;
subplot(3,1,1);
stem(n,delta,'filled');
title('Unit Impulse Sequence');
xlabel('n');
ylabel('Amplitude');

subplot(3,1,2);
stem(n,u,'filled');
title('Unit Step Sequence');
xlabel('n');
ylabel('Amplitude');

subplot(3,1,3);
stem(n,r,'filled');
title('Ramp Sequence');
xlabel('n');
ylabel('Amplitude');
```

RESULT: Thus, we have generated various basic discrete sequences using MATLAB.

EXPERIMENT NO. 03

NAME OF EXPERIMENT: - To perform basic operations on discrete-time signals (Addition, Subtraction, Multiplication and Scaling).

OBJECTIVES:

1. To perform addition, subtraction, and multiplication of discrete-time signals using MATLAB.
2. To visualize and analyze the resulting signals using discrete-time stem plots.

SOFTWARE: MATLAB V 7.7

PROGRAM:

```
clc;
clear;
close all;
% Input signals
x1 = input('Enter first signal = ');
x2 = input('Enter second signal = ');
% Length of signals
N1 = length(x1);
N2 = length(x2);
% Make both signals equal length by adding zeros
N = max(N1,N2);
x1(N) = 0;
x2(N) = 0;
% Signal operations
add = x1 + x2;
sub = x1 - x2;
mul = x1 .* x2;
% Display results
disp('First Signal = ');
disp(x1);
disp('Second Signal = ');
disp(x2);
disp('Addition = ');
disp(add);
disp('Subtraction = ');
disp(sub);
disp('Multiplication = ');
disp(mul);
% Plot signals
n = 0:N-1;
figure;
```

```
subplot(5,1,1);
stem(n,x1,'filled');
title('First Signal');
xlabel('n');
ylabel('Amplitude');
grid on;

subplot(5,1,2);
stem(n,x2,'filled');
title('Second Signal');
xlabel('n');
ylabel('Amplitude');
grid on;

subplot(5,1,3);
stem(n,add,'filled');
title('Addition');
xlabel('n');
ylabel('Amplitude');
grid on;

subplot(5,1,4);
stem(n,sub,'filled');
title('Subtraction');
xlabel('n');
ylabel('Amplitude');
grid on;

subplot(5,1,5);
stem(n,mul,'filled');
title('Multiplication');
xlabel('n');
ylabel('Amplitude');
grid on;
```

Input:

Enter first signal = [1 2 3 4]

Enter second signal = [7 9 2 11]

RESULT: Thus, we have basic signals operations using MATLAB.

EXPERIMENT NO: 04

NAME OF EXPERIMENT: To perform linear convolution of discrete-time signals using MATLAB.

OBJECTIVES:

1. To perform linear convolution of two discrete-time sequences using MATLAB with and without the built-in conv() function.
2. To understand the step-by-step implementation of linear convolution using nested loops and analyze the resulting output sequence.

SOFTWARE: MATLAB V 7.7

PROGRAM:

```
clc;
clear;
close all;
% With Conv command
x = input('Enter first sequence: ');
h = input('Enter second sequence: ');
y = conv(x,h);
disp('Linear Convolution Output = ');
disp(y);
stem(y,'filled');
title('Linear Convolution');
xlabel('Samples');
ylabel('Amplitude');
grid on;

% Without Conv command
% Length of input sequences
Nx = length(x);
Nh = length(h);
% Length of output sequence
Ny = Nx + Nh - 1;
% Initialize output
y = zeros(1, Ny);
% Linear convolution using nested loops
for n = 1:Ny
  for k = 1:Nx
    j = n - k + 1;
    if j >= 1 && j <= Nh
      y(n) = y(n) + x(k) * h(j);
    end
  end
end
```

```
% Display results
disp('Input sequence x(n):');
disp(x);
disp('Impulse response h(n):');
disp(h);
disp('Linear convolution y(n):');
disp(y);
figure
% Plot sequences
subplot(3,1,1);
stem(x);
title('Input Sequence x(n)');
xlabel('n');
ylabel('x(n)');

subplot(3,1,2);
stem(h);
title('Impulse Response h(n)');
xlabel('n');
ylabel('h(n)');

subplot(3,1,3);
stem(y);
title('Linear Convolution y(n)');
xlabel('n');
ylabel('y(n)');
```

INPUT:

Enter first sequence: [1 2 3 4]
Enter second sequence: [1 2 3 4]

RESULT: Thus we have performed Linear Convolution using both conv and without conv command.

EXPERIMENT NO. 05

NAME OF EXPERIMENT: To analyze the Region of Convergence (ROC) for different discrete-time signals.

OBJECTIVES:

1. To generate and plot right-sided, left-sided, and two-sided discrete-time exponential signals using MATLAB.
2. To determine and understand the Region of Convergence (ROC) for different types of signals in the Z-domain.

SOFTWARE: MATLAB V 7.7

PROGRAM:

```
clc;
clear;
close all;
%% Right-sided Signal
n1 = 0:10;
a = 0.5;
xr = a.^n1;

%% Left-sided Signal
n2 = -10:0;
xl = a.^(-n2);

%% Two-sided Signal
n3 = -10:10;
xt = a.^abs(n3);

%% Plotting
figure('Name','Region of Convergence (ROC)','NumberTitle','off');

subplot(3,1,1)
stem(n1,xr,'filled','LineWidth',1.5)
grid on
title('Right-sided Signal')
xlabel('n')
ylabel('Amplitude')
```

```
subplot(3,1,2)
stem(n2,xl,'filled','LineWidth',1.5)
grid on
title('Left-sided Signal')
xlabel('n')
ylabel('Amplitude')

subplot(3,1,3)
stem(n3,xt,'filled','LineWidth',1.5)
grid on
title('Two-sided Signal')
xlabel('n')
ylabel('Amplitude')

%% Display ROC
disp('-----------------------------------')
disp('REGION OF CONVERGENCE (ROC)')
disp('-----------------------------------')
disp('1. Right-sided Signal')
disp('ROC : |z| > 0.5')
disp(' ')
disp('2. Left-sided Signal')
disp('ROC : |z| < 0.5')
disp(' ')
disp('3. Two-sided Signal')
disp('ROC : 0.5 < |z| < Infinity')
```

RESULT: Thus we have studied ROC on different signals.

EXPERIMENT NO: 06

NAME OF EXPERIMENT: To perform circular convolution using FFT/IFFT.

OBJECTIVES:

1. To perform circular convolution of two discrete-time sequences using the DFT and IDFT in MATLAB.
2. To understand the use of FFT and IFFT for efficient computation of circular convolution.

SOFTWARE: MATLAB V 7.7

PROGRAM:

```
clc;
clear;
close all;
x = input('Enter first sequence: ');
h = input('Enter second sequence: ');
N = max(length(x),length(h));
x = [x zeros(1,N-length(x))];
h = [h zeros(1,N-length(h))];
Y = ifft(fft(x).*fft(h));
disp('Circular Convolution Output');
disp(real(Y));
stem(real(Y),'filled');
title('Circular Convolution');
xlabel('Samples');
ylabel('Amplitude');
grid on;
```

INPUT:

```
Enter first sequence: [1 2 3 4]
Enter second sequence: [1 2 3 4]
```

RESULT: Thus, we have performed Circular convolution.

EXPERIMENT NO: 07

NAME OF EXPERIMENT: To implement the Radix-2 Decimation-in-Time (DIT) FFT algorithm using MATLAB.

OBJECTIVES:

1. To compute the Fast Fourier Transform (FFT) of a given discrete-time sequence using MATLAB.
2. To obtain and analyze the magnitude and phase spectra of the input sequence.

SOFTWARE: MATLAB

PROGRAM:

```
clc;
clear;
close all;
% Input sequence
x = input('Enter the input sequence: ');
% Length of signal
N = length(x);
% Compute FFT
X = fft(x);
% Display results
disp('Input Sequence:');
disp(x);
disp('FFT Output:');
disp(X);
% Magnitude and Phase
mag = abs(X);
phase = angle(X);
% Frequency index
k = 0:N-1;
% Plot Results
figure('Name','FFT Analysis','NumberTitle','off');

subplot(3,1,1)
stem(k,x,'filled','LineWidth',1.5)
title('Input Sequence')
xlabel('Sample Index')
ylabel('Amplitude')
grid on

subplot(3,1,2)
stem(k,mag,'filled','LineWidth',1.5)
title('Magnitude Spectrum')
xlabel('Frequency Index')
```

```
ylabel('|X(k)|')
grid on

subplot(3,1,3)
stem(k,phase,'filled','LineWidth',1.5)
title('Phase Spectrum')
xlabel('Frequency Index')
ylabel('Phase (Radians)')
grid on
```

INPUT:

Enter the input sequence: [1 2 3 4]

RESULT: Thus, we have performed Fast Fourier Transform.

EXPERIMENT NO: 08

NAME OF EXPERIMENT: To design a digital filter using the Rectangular Window Method.

OBJECTIVES:

1. To design an FIR Low-Pass Filter using a Rectangular Window in MATLAB.
2. To analyze the frequency response of the designed FIR filter using MATLAB.

SOFTWARE: MATLAB V 7.7

PROGRAM:

```
clc;
clear;
close all;
% Filter Specifications
N = 11;         % Filter Order
Fc = 0.4;       % Normalized Cutoff Frequency (0 to 1)
% Design FIR Low-Pass Filter using Rectangular Window
b = fir1(N,Fc,'low',rectwin(N+1));
% Display Filter Coefficients
disp('Filter Coefficients are:')
disp(b)
% Frequency Response
figure;
freqz(b,1);
title('FIR Low Pass Filter using Rectangular Window');
grid on;
```

Result: The FIR Low-Pass Filter using a Rectangular Window was successfully designed, and its frequency response was obtained and plotted using MATLAB.

EXPERIMENT NO: 09

NAME OF EXPERIMENT: To perform Sampling Rate Conversion (Interpolation and Decimation)

OBJECTIVES:

1. To perform interpolation (up-sampling) and decimation (down-sampling) of a discrete-time signal using MATLAB.
2. To observe and analyze the effect of sampling-rate conversion on the discrete-time signal.

SOFTWARE: MATLAB V 7.7

PROGRAM:

```
clc;
clear;
close all;
x = [1 2 3 4 5];
disp('Original Signal');
disp(x);
% Interpolation (Upsampling by 2)
y = upsample(x,2);
disp('Interpolated Signal');
disp(y);
% Decimation by 2
z = downsample(y,2);
disp('Decimated Signal');
disp(z);

figure
subplot(3,1,1)
stem(x,'filled')
title('Original Signal')
grid on

subplot(3,1,2)
stem(y,'filled')
title('Interpolated Signal (Upsampling by 2)')
grid on

subplot(3,1,3)
stem(z,'filled')
title('Decimated Signal')
grid on
```

Result: Thus, we have studied concept of Interpolation and Decimation.

EXPERIMENT NO: 10

NAME OF EXPERIMENT: To Find Cross-Correlation of Two Discrete-Time Signals Using Convolution

OBJECTIVES:

1. To understand the concept of cross-correlation between two discrete-time signals.
2. To implement cross-correlation using the relationship between convolution and time reversal of a signal in MATLAB

SOFTWARE: MATLAB

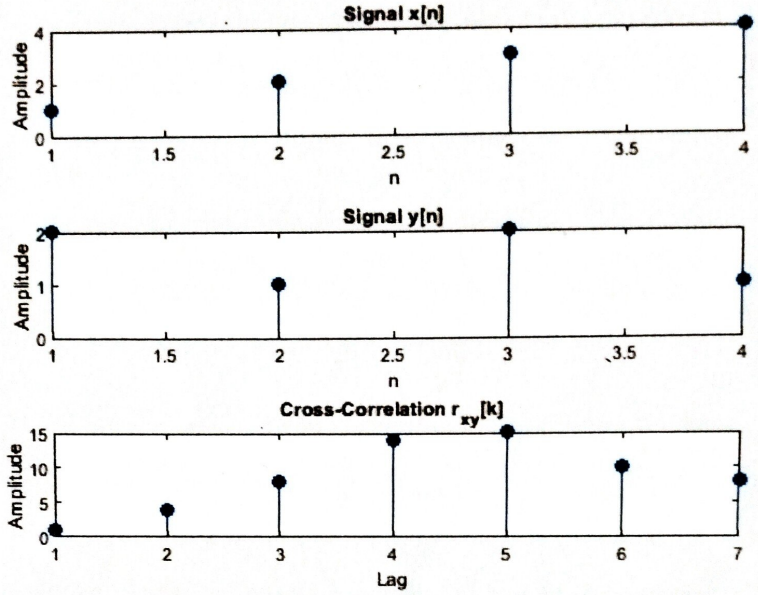
PROGRAM:

```
clc;
clear;
close all;
% Input signals
x = [1 2 3 4];
y = [2 1 2 1];
% Reverse the second signal
y_reverse = fliplr(y);
% Correlation using convolution
rxy = conv(x, y_reverse);
% Display results
disp('Signal x:');
disp(x);
disp('Signal y:');
disp(y);
disp('Reversed signal y:');
disp(y_reverse);
disp('Cross-correlation using convolution:');
disp(rxy);
% Plot
figure;
subplot(3,1,1);
stem(x, 'filled');
title('Signal x[n]');
xlabel('n');
ylabel('Amplitude');
subplot(3,1,2);
stem(y, 'filled');
title('Signal y[n]');
xlabel('n');
ylabel('Amplitude');
subplot(3,1,3);
stem(rxy, 'filled');
title('Cross-Correlation r_{xy}[k]');
xlabel('Lag');
ylabel('Amplitude');
```

Result: The cross-correlation of the given discrete-time signals was successfully obtained using convolution in MATLAB.

Output:

Signal x: 1 2 3 4
Signal y: 2 1 2 1
Reversed signal y: 1 2 1 2
Cross-correlation using convolution: 1 4 8 14 15 10 8



Result: The cross-correlation of the given discrete-time signals was successfully obtained using convolution in MATLAB.